Week 9: Sales and Customer Analytics System

LOGESHWARI D(E25AI121)

Scenario:

The supermarket management system analyzes sales data to understand customer purchasing behaviour, product performance, and revenue trends. The management team needs reports to identify top customers, best-selling products, and profitable product categories.

The Week 9 report will cover:

1. Apply SQL Aggregate Functions
2. Generate Total Sales Reports
3. Customer Purchase Analysis
4. Find Top Customers Based on Purchase Amount
5. Identify Best-Selling Products
6. Perform Category-wise Sales Analysis
7. Generate Business Analytics Reports

1. Generate Total Sales Report

Query 1: Count the total number of sales

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM SALES;

TOTAL_ORDERS
------------
           5
```

Query 2: Calculate total revenue

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM SALES;

TOTAL_REVENUE
-------------
         1885
```

Query 3: Calculate average sale amount

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_SALE
  2  FROM SALES;

AVERAGE_SALE
------------
         377
```

Query 4: Find the highest and lowest order amount

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER,
  2         MIN(TOTAL_AMOUNT) AS LOWEST_ORDER
  3  FROM SALES;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
          470          300
```

Query 5: Find total sales for a specific period

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS PERIOD_SALES
  2  FROM SALES
  3  WHERE SALE_DATE BETWEEN
  4  TO_DATE('01-10-2026','DD-MM-YYYY')
  5  AND TO_DATE('30-10-2026','DD-MM-YYYY');

PERIOD_SALES
------------
```

2.Customer Purchase Analysis

1: Find the total number of orders placed by each customer

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      COUNT(S.SALE_ID) AS TOTAL_ORDERS
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_ORDERS
----------- ------------------------------ ------------
          2 Khilan                                    1
          1 Ramesh                                    1
          5 Hardik                                    1
          3 kaushik                                   1
          4 Chaitali                                  1
```

2. Total amount spent by each customer.

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      NVL(SUM(S.TOTAL_AMOUNT), 0) AS TOTAL_AMOUNT_SPENT
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  ORDER BY TOTAL_AMOUNT_SPENT DESC;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_AMOUNT_SPENT
----------- ------------------------------ ------------------
          4 Chaitali                                      470
          1 Ramesh                                        415
          5 Hardik                                        360
          3 kaushik                                       340
          2 Khilan                                        300
```

3. Average spending per customer.

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      NVL(AVG(S.TOTAL_AMOUNT), 0) AS AVERAGE_SPENDING
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  ORDER BY C.ID;

CUSTOMER_ID CUSTOMER_NAME                  AVERAGE_SPENDING
----------- ------------------------------ ----------------
          1 Ramesh                                      415
          2 Khilan                                      300
          3 kaushik                                     340
          4 Chaitali                                    470
          5 Hardik                                      360
```

4. Find the customer with the maximum purchase amount.

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      S.TOTAL_AMOUNT AS MAXIMUM_PURCHASE_AMOUNT
  5  FROM CUSTOMER C
  6  JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  WHERE S.TOTAL_AMOUNT =
  9      (SELECT MAX(TOTAL_AMOUNT)
 10       FROM SALES);

CUSTOMER_ID CUSTOMER_NAME                  MAXIMUM_PURCHASE_AMOUNT
----------- ------------------------------ -----------------------
          4 Chaitali                                           470
```

5. Find customers with fewer purchases.

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      COUNT(S.SALE_ID) AS TOTAL_ORDERS
  5  FROM CUSTOMER C
  6  LEFT JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  ORDER BY TOTAL_ORDERS ASC;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_ORDERS
----------- ------------------------------ ------------
          2 Khilan                                    1
          1 Ramesh                                    1
          5 Hardik                                    1
          3 kaushik                                   1
          4 Chaitali                                  1
```

6. Find the top 5 customers based on purchase amount

1.Top 5 customers based on total spending

```
SQL> SELECT *
  2  FROM
  3  (
  4      SELECT
  5          C.ID AS CUSTOMER_ID,
  6          C.NAME AS CUSTOMER_NAME,
  7          SUM(S.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT
  8      FROM CUSTOMER C
  9      JOIN SALES S
 10          ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
 11      GROUP BY
 12          C.ID,
 13          C.NAME
 14      ORDER BY TOTAL_PURCHASE_AMOUNT DESC
 15  )
 16  WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_PURCHASE_AMOUNT
----------- ------------------------------ ---------------------
          4 Chaitali                                       470
          1 Ramesh                                         415
          5 Hardik                                         360
          3 kaushik                                        340
          2 Khilan                                         300
```

2.Find customers with the maximum number of orders.

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      COUNT(S.SALE_ID) AS TOTAL_ORDERS
  5  FROM CUSTOMER C
  6  JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  HAVING COUNT(S.SALE_ID) =
 12  (
 13      SELECT MAX(ORDER_COUNT)
 14      FROM
 15      (
 16          SELECT COUNT(*) AS ORDER_COUNT
 17          FROM SALES
 18          GROUP BY CUSTOMER_ID
 19      )
 20  );

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_ORDERS
----------- ------------------------------ ------------
          2 Khilan                                    1
          1 Ramesh                                    1
          4 Chaitali                                  1
          3 kaushik                                   1
          5 Hardik                                    1
```

3. Find frequent customers.

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      COUNT(S.SALE_ID) AS TOTAL_ORDERS
  5  FROM CUSTOMER C
  6  JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  HAVING COUNT(S.SALE_ID) > 1
 12  ORDER BY TOTAL_ORDERS DESC;

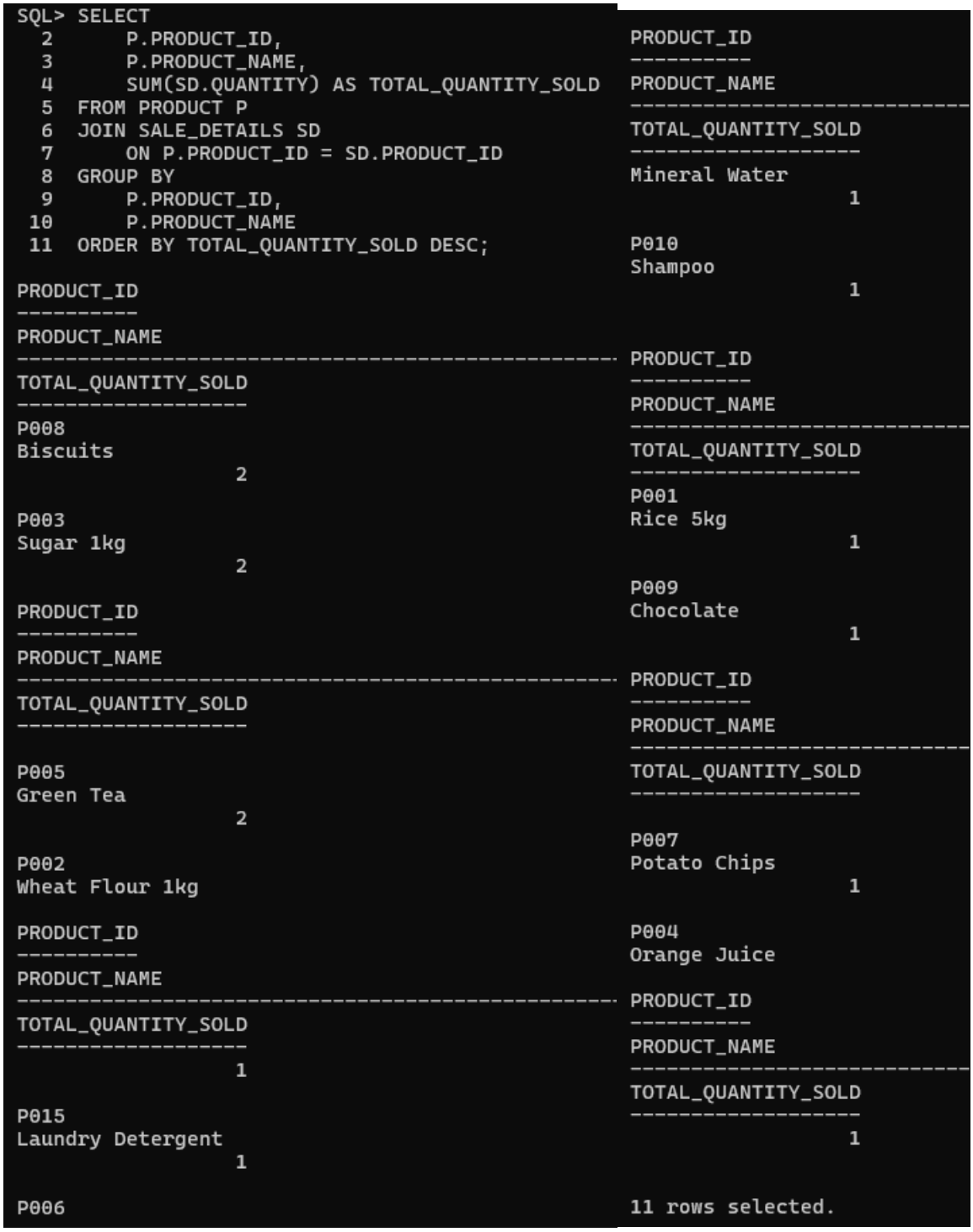
no rows selected
```

4.Identify High-Value Customers

```
SQL> SELECT
  2      C.ID AS CUSTOMER_ID,
  3      C.NAME AS CUSTOMER_NAME,
  4      SUM(S.TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM CUSTOMER C
  6  JOIN SALES S
  7      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  8  GROUP BY
  9      C.ID,
 10      C.NAME
 11  HAVING SUM(S.TOTAL_AMOUNT) >
 12  (
 13      SELECT AVG(TOTAL_SPENDING)
 14      FROM
 15      (
 16          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
 17          FROM SALES
 18          GROUP BY CUSTOMER_ID
 19      )
 20  )
 21  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_SPENDING
----------- ------------------------------ --------------
          4 Chaitali                                  470
          1 Ramesh                                    415
```

5. Identify Best-Selling Products



```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      SUM(SD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT P
  6  JOIN SALE_DETAILS SD
  7      ON P.PRODUCT_ID = SD.PRODUCT_ID
  8  GROUP BY
  9      P.PRODUCT_ID,
 10      P.PRODUCT_NAME
 11  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P008
Biscuits
                  2

P003
Sugar 1kg
                  2

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P005
Green Tea
                  2

P002
Wheat Flour 1kg

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1

P015
Laundry Detergent
                  1

P006
Mineral Water
                  1

P010
Shampoo
                  1


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P001
Rice 5kg
                  1

P009
Chocolate
                  1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P007
Potato Chips
                  1

P004
Orange Juice

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1


11 rows selected.
```

6. Identify Products Generating the Highest Revenue

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      SUM(SD.QUANTITY * SD.PRICE) AS TOTAL_REVENUE
  5  FROM PRODUCT P
  6  JOIN SALE_DETAILS SD
  7      ON P.PRODUCT_ID = SD.PRODUCT_ID
  8  GROUP BY
  9      P.PRODUCT_ID,
 10      P.PRODUCT_NAME
 11  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
TOTAL_REVENUE
-------------
P005
Green Tea
          360

P001
Rice 5kg
          350

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
TOTAL_REVENUE
-------------

P015
Laundry Detergent
          280

P010
Shampoo

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
TOTAL_REVENUE
-------------
          220

P004
Orange Juice
          120

P003
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------
TOTAL_REVENUE
-------------
Sugar 1kg
          110

P009
Chocolate
           80


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------
TOTAL_REVENUE
-------------
P002
Wheat Flour 1kg
           65

P008
Biscuits
           60

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------
TOTAL_REVENUE
-------------

P007
Potato Chips
           40

P006
Mineral Water

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------
TOTAL_REVENUE
-------------
           25


11 rows selected.
```

7.Identify Least-Selling Products

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(SD.QUANTITY), 0) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT P
  6  LEFT JOIN SALE_DETAILS SD
  7      ON P.PRODUCT_ID = SD.PRODUCT_ID
  8  GROUP BY
  9      P.PRODUCT_ID,
 10      P.PRODUCT_NAME
 11  ORDER BY TOTAL_QUANTITY_SOLD ASC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P013
Dishwash Liquid
                  0

P014
Floor Cleaner
                  0

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P011
Bath Soap
                  0

P012
Toothpaste

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  0

P006
Mineral Water
                  1

P004
```



8. Identify Products Requiring Promotion

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(SD.QUANTITY), 0) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT P
  6  LEFT JOIN SALE_DETAILS SD
  7      ON P.PRODUCT_ID = SD.PRODUCT_ID
  8  GROUP BY
  9      P.PRODUCT_ID,
 10      P.PRODUCT_NAME
 11  HAVING NVL(SUM(SD.QUANTITY), 0) <= 2
 12  ORDER BY TOTAL_QUANTITY_SOLD ASC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P013
Dishwash Liquid
                  0

P014
Floor Cleaner
                  0

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P011
Bath Soap
                  0

P012
Toothpaste

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  0

P006
Mineral Water
                  1

P004

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
Orange Juice
                  1

P002
Wheat Flour 1kg
                  1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P001
Rice 5kg
                  1

P007
Potato Chips
                  1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P009
Chocolate
                  1

P015
Laundry Detergent

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1

P010
Shampoo
                  1

P005

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
Green Tea
                  2

P003
Sugar 1kg
                  2


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P008
Biscuits
                  2


15 rows selected.
```

9. Calculate Total Sales Generated by Each Category

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(SUM(SD.QUANTITY * SD.PRICE), 0) AS TOTAL_SALES
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  LEFT JOIN SALE_DETAILS SD
  9      ON P.PRODUCT_ID = SD.PRODUCT_ID
 10  GROUP BY
 11      C.CATEGORY_ID,
 12      C.CATEGORY_NAME
 13  ORDER BY TOTAL_SALES DESC;

CATEGORY_I CATEGORY_NAME                                      TOTAL_SALES
---------- -------------------------------------------------- -----------
CAT001     Grocery                                                    525
CAT002     Beverages                                                  505
CAT005     Household                                                  280
CAT004     Personal Care                                              220
CAT003     Snacks                                                     180
```

10.Calculate Number of Products Sold in Each Category

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(SUM(SD.QUANTITY), 0) AS TOTAL_PRODUCTS_SOLD
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  LEFT JOIN SALE_DETAILS SD
  9      ON P.PRODUCT_ID = SD.PRODUCT_ID
 10  GROUP BY
 11      C.CATEGORY_ID,
 12      C.CATEGORY_NAME
 13  ORDER BY TOTAL_PRODUCTS_SOLD DESC;

CATEGORY_I CATEGORY_NAME
---------- --------------------------------------------------
TOTAL_PRODUCTS_SOLD
-------------------
CAT001     Grocery
                   4

CAT002     Beverages
                   4

CAT003     Snacks
                   4


CATEGORY_I CATEGORY_NAME
---------- --------------------------------------------------
TOTAL_PRODUCTS_SOLD
-------------------
CAT004     Personal Care
                   1

CAT005     Household
                   1
```

11. Identify the Highest Revenue-Generating Category

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      SUM(SD.QUANTITY * SD.PRICE) AS TOTAL_REVENUE
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  JOIN SALE_DETAILS SD
  9      ON P.PRODUCT_ID = SD.PRODUCT_ID
 10  GROUP BY
 11      C.CATEGORY_ID,
 12      C.CATEGORY_NAME
 13  HAVING SUM(SD.QUANTITY * SD.PRICE) =
 14  (
 15      SELECT MAX(CATEGORY_REVENUE)
 16      FROM
 17      (
 18          SELECT SUM(SD2.QUANTITY * SD2.PRICE) AS CATEGORY_REVENUE
 19          FROM PRODUCT P2
 20          JOIN SALE_DETAILS SD2
 21              ON P2.PRODUCT_ID = SD2.PRODUCT_ID
 22          GROUP BY P2.CATEGORY_ID
 23      )
 24  );

CATEGORY_I CATEGORY_NAME                                      TOTAL_REVENUE
---------- -------------------------------------------------- -------------
CAT001     Grocery                                                      525
```

12. Calculate Average Sales per Category

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(AVG(SD.QUANTITY * SD.PRICE), 0) AS AVERAGE_SALES
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  LEFT JOIN SALE_DETAILS SD
  9      ON P.PRODUCT_ID = SD.PRODUCT_ID
 10  GROUP BY
 11      C.CATEGORY_ID,
 12      C.CATEGORY_NAME
 13  ORDER BY AVERAGE_SALES DESC;

CATEGORY_I CATEGORY_NAME                                      AVERAGE_SALES
---------- -------------------------------------------------- -------------
CAT005     Household                                                    280
CAT004     Personal Care                                                220
CAT001     Grocery                                                      175
CAT002     Beverages                                                 126.25
CAT003     Snacks                                                       45
```

3.Generating Business Analytics report

1.Sales performance report

*Total sales

*Number of orders

*Average Order Value*highest order value

```
SQL> SELECT
  2      SUM(TOTAL_AMOUNT) AS TOTAL_SALES,
  3      COUNT(SALE_ID) AS NUMBER_OF_ORDERS,
  4      MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE,
  5      ROUND(AVG(TOTAL_AMOUNT), 2) AS AVERAGE_ORDER_VALUE
  6  FROM SALES;

TOTAL_SALES NUMBER_OF_ORDERS HIGHEST_ORDER_VALUE AVERAGE_ORDER_VALUE
----------- ---------------- ------------------- -------------------
       1885                5                 470                 377
```

2.Customer analytics report

*Customer Name

*Number of Orders

*Total Spending

*Purchase Frequency

```
SQL> SELECT
  2      C.NAME AS CUSTOMER_NAME,
  3      COUNT(S.SALE_ID) AS NUMBER_OF_ORDERS,
  4      NVL(SUM(S.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING,
  5      COUNT(S.SALE_ID) AS PURCHASE_FREQUENCY
  6  FROM CUSTOMER C
  7  LEFT JOIN SALES S
  8      ON 'C' || LPAD(C.ID, 3, '0') = S.CUSTOMER_ID
  9  GROUP BY
 10      C.ID,
 11      C.NAME
 12  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_SPENDING
------------------------------ ---------------- --------------
PURCHASE_FREQUENCY
------------------
Chaitali                                      1            470
                 1

Ramesh                                        1            415
                 1

Hardik                                        1            360
                 1


CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_SPENDING
------------------------------ ---------------- --------------
PURCHASE_FREQUENCY
------------------
kaushik                                       1            340
                 1

Khilan                                        1            300
                 1
```

3.Products performance report

*Product name

*Quantity sold

*Revenue generated

```
SQL> SELECT
  2      P.PRODUCT_NAME,
  3      SUM(SD.QUANTITY) AS QUANTITY_SOLD,
  4      SUM(SD.QUANTITY * SD.PRICE) AS REVENUE_GENERATED
  5  FROM PRODUCT P
  6  JOIN SALE_DETAILS SD
  7      ON P.PRODUCT_ID = SD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  ORDER BY REVENUE_GENERATED DESC;

PRODUCT_NAME
--------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
Green Tea
            2               360

Rice 5kg
            1               350

Laundry Detergent
            1               280


PRODUCT_NAME
--------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
Shampoo
            1               220

Orange Juice
            1               120

Sugar 1kg
            2               110


PRODUCT_NAME
--------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
Chocolate
            1                80

Wheat Flour 1kg
            1                65

Biscuits
            2                60


PRODUCT_NAME
--------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
Potato Chips
            1                40

Mineral Water
            1                25


11 rows selected.
```

4.Category Analysis Report

*Category Name

*Total Products sold

*Total Revenue

```
SQL> SELECT
  2      C.CATEGORY_NAME,
  3      SUM(SD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  4      SUM(SD.QUANTITY * SD.PRICE) AS TOTAL_REVENUE
  5  FROM CATEGORY C
  6  JOIN PRODUCT P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  JOIN SALE_DETAILS SD
  9      ON P.PRODUCT_ID = SD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Grocery                                                              4
          525

Beverages                                                            4
          505

Household                                                            1
          280


CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Personal Care                                                        1
          220

Snacks                                                               4
          180
```

Conclusion

The DBMS project successfully analyzes sales and product information using SQL queries. The Products Performance Report identifies product-wise quantities sold and revenue generated, while the Category Analysis Report summarizes total products sold and revenue for each category. These reports help identify high-performing products and profitable categories, monitor sales performance, and support better business decisions. Overall, the project demonstrates how a database management system can organize, retrieve, and analyze data efficiently to improve business operations.